

Shared Memory, Semaphores and Message Queues ([Previous Notes](#))

These three seeming unrelated topics are presented together in they are based on a common structure.

- These system calls are referred to as IPC (inter process communications) calls.
- The IPC calls are unusual in that they are **not dependent on the file system infrastructure**. This differs from so many other system elements which are tied to files descriptors.
- **Three services are provided through IPC - shared memory, semaphores, and message queues.** The most important of these 3 services is shared memory. We might skip at 1 of them. Namely Semaphores in that there are newer versions of this. OTOH, the more understanding you have of this structure, the more effectively you can work.
- **Note: there are alternatives to shared memory as described here. We will discuss later if there is time.**
- **keys are used to name and access the IPC structures.** A key is a positive integer. A key is used to name an IPC structure in the operating system.
- **From the key, you can get an ID (An integer) which most system calls need to reference the IPC structures.** Note: IDs are different from file descriptors in that they are global. Seems like a weird setup to use two different global names for a structure. This is an example of a design that could have been done better. Why don't they change it? In truth, having a random key and a better behaved ID might not be so bad.
- Note: IPC structures are persistent. What does that mean? Can this be a problem? (YES!)

Basic commands

There are two commands associated with IPC structures. The first, **ipcs**, is used to get general information (status) about IPC structures. (As we will see, besides us, no one on our machine tends to use IPC structures.) The second, **ipcrm**, is used to remove IPC structures. **It is important to remove them or they stay around.** You must make sure that you do this if you allocate such a structure, in that some of these resources are limited.

The following is a description of each command with a basic set of options.

`ipcs [-m] [-q] [-s] [-a]` (-m = shared memory, -q = message queues, -s = semaphores, -a = list all information This last option doesn't change anything.)

The default for `ipcs` is to list everything. So why have the -a option. We will

use this command to see what is happening on our system with respect to these three IPC structures.

```
ipcrm [-m shmid] [-q msgid] [-s semid] [-M shmkey] [-Q msgkey] [-S semkey]
```

Each IPC structure is referred in systems calls by an identifier. (ID) This is a nonnegative integer. The identifier is assigned to the structure when it is created. It is also referenced by a key. [A very strange thing to have two global names for the same structure.](#) (Yes, I am repeating myself)

Introductory Material.

First some initial ideas:

- [When an IPC structure is created or accessed, a key must be used.](#)
- This is of type `key_t` (usually this is a long int, but no guarantee).
- The `shmget`, `semget`, and `msgget` functions (recall that there are 3 IPC structures) are used to create and prepare to access an IPC structure. They have a key as an argument. [These return the IPC identifier \(ID\)](#), which is used for all other calls.
- [Why have both a key and an ID? Yes I am repeating. \(Boring\)](#)
- IPC structures have owners and groups. They have permissions are similar to file permissions. This can be very helpful for controlling access..
- For example, shared memory may be set up so that it has read/write access for owner processes and it has read only by any process that does not belong to the owner of the shared memory. (This is assuming that the operating system/hardware supports access control to memory.)
- Read book on `IPC_PRIVATE` (value of 0) as a key. For shared memory, it creates a shared memory segment that can only be accessed by the calling process and its children. BTW, it still has to be removed before these processes terminate. Or it has to be removed using the `ipcrm` command.
- Using IPC structures requires some management. [BIG problems can occur if two independent processes chose the same key.](#) Having restrictive permissions can help prevent a big disaster. What else can be done?
- Keys values must be reserved in some consistent way or every project must be required to use the `ftok` function to generate a key. The `ftok` function generates a unique key from a pathname and a one-byte project id. Of course, everyone must agree to use file names to generate keys.
- For large systems with many applications, a good tactic is to keep a write-protected directory for file names to be used by `ftok`.

The `ftok` system call

The format of the `ftok` system call is as follows:

```
#include <sys/types.h>
#include <sys/ipc.h>
key_t ftok( char *path, char id );
```

Returns a unique key if successful and (key_t) -1 if it fails. It fails if the path/file does not exist.

Shared Memory

- We will start with shared memory. This is the most widely used of the 3 structures.
- All the other IPC structures have similar system calls.
- Shared memory is memory that is accessible to multiple processes.
- By orders of magnitude, it is the quickest way of sharing large amounts of information among a set of processes.
- Keep in mind that shared memory is available on all operating systems that I know of. Only the calls to create and access shared memory will be different.

Note: there are alternative ways of having shared memory on Linux. [See memory maps which we may discuss later.](#)

Examples of use:

1. To implement very big pipes whose status and content may be viewed by other processes. Recall that pipes were only 64K in Linux.
2. To share current day's quote data for a large number of stocks among a bunch of processes. This can be billions of quotes in a day.
3. To maintain dynamically updated statistics on a set of running processes. This is great in that you could have an additional program that dynamically displays the statistics and any error events.

[In class, we will discuss how shared memory is implemented in an operating system.](#)

This hopefully will be a review of what you learned in operating systems.

Note: shared memory is persistent. It does not go away when no program is referencing it. This can be a good thing, but it can tie up system resources. Please [make sure that you clean up after yourself](#). This is why the `ipcrm` command is important. [I will be annoyed if I have to keep having to remind you to clean up after yourself.](#) Be careful not to clean up the resources allocated to NoMachine (If you use it) for you and also Eclipse. That is why I will suggest that you use strange permission settings.

The `shmget` system call

shmget system call. Gets and/or creates a shared memory segment. The call has the following format:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget ( key_t key, size_t size, int shmflg)
```

Returns shared memory ID if successful and -1 if error. The ID, as its name implies, identifies the shared memory segment and is used in other calls.

key has been discussed before. It identifies this IPC tool.

size is used when creating the shared memory segment to specify the size. If the shared memory segment exists, it may be 0 or used to specify the minimum size of the shared memory. The size of the shared memory you request is less than or equal to the amount you actually get. That is, due to the fact that shared memory is allocated in pages, so, the actual amount of shared memory allocated may be bigger than the size requested.

shmflg are flags specifying options for this command. Right-most 9 bits are permissions. (Same bits as in the open call) SHM_R and SHM_W for owner and these shifted 3 bits to the right for group and these shifted 6 bits to the right for other. Also have IPC_CREAT and IPC_EXCL bits. These are similar to O_CREAT and O_EXCL for the file system. There are additional flags that are not as general. For example, on Solaris, there is the ability to make the shared memory dynamically resizable. This feature was not available on all UNIX systems.

The maximum size of shared memory can be system dependent. Ours is huge.

Note: shmget does not make the memory available to your program. You need another call to do this.

shmat system call

Attaches the shared memory segment to our process. This is separate from shmget because there is a limit on the number of shared memory segments to which we can attach. This is a system-imposed limit. It used to be as small as two. [\(Let's try and figure this out when I do an example. \)](#) So, if you are dealing with a very large number of shared memory segments, a good strategy would be to get the IDs for all that we need and then attach only to those shared memory segments that we are actively using. We can detach from a shared memory segment.

The system call shmat is defined as follows:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
void *shmat ( int shmid, const void *shmaddr, int shmflg );
```

Returns a **pointer** to the shared memory segment or -1 if error.

shmid is the shared memory ID

shmaddr should be NULL. This argument can allow you to specify the address to associate with the shared memory. For modern computers, it is hard to find a reason for using a non-NULL argument. **Do not specify an address unless you have a very good reason and know what you are doing.** I have never had a good reason to do this.

shmflg are flags for this call. The only one that we will care about is SHM_RDONLY. This makes the shared memory segment read only. This is like opening a file to be read only. So, if we don't want read only shared memory, this argument will be set to zero. Having this flag set is a good way of proving that you program did not mess up the shared memory.

The shmdt system call

shmdt detaches a shared memory segment from a process. As mentioned earlier, there is a limit to how many shared memory segments you may attach to. shmdt may be have to be used so that another segment may be attached. The format of this call is as follows:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
int shmdt (const void *shmaddr );
```

shmaddr is the address of the shared memory segment.

Returns 0 if successful and -1 if fails.

shmctl system call

shmctl performs a bunch of utility functions on shared memory. Its description follows:

```
#include <sys/types.h>
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
int shmctl ( int shmid, int cmd, struct shmid_ds *buf)
```

Returns 0 if successful and -1 if fails.

cmd is the command that we wish to have executed.

IPC_RMID removes the shared memory segment when no one is using it. What does that mean?

IPC_STAT gets data associated with shared memory segment into buf

IPC_SET allows 3 fields in buf to be changed.

buf is used by certain commands. The most often used command is IPC_RMID.

See example for much clarification. [shm1.cpp](#), [shm2.cpp](#) The first piece of code writes something to shared memory and the second retrieves it.

Notes on example:

1. Uncomment the code and show that the shared memory can be deleted.
2. Use ipcrm to remove the shared memory while the program is still attached to it. Do this in the debugger so you can see that the shared memory did not go away. The shared memory does not show up but it is there. Prove it by removing it and have the program call shmdt and look at it again.
3. Show that ftok will give a different key if the file is removed and added.
4. Show that readonly permission works.
5. [Show that the shared memory, when deleted does not go away until everyone is detached.](#)

If there is a lot of information to be recorded in shared memory, a struct is used to hold the data. For example:

```
#define STUDENTS_SIZE 2056
typedef struct {
    long int date;
    struct students st[STUDENTS_SIZE];
    int duck; /* There always has to be a duck. */
} SharedMemory;
```

```
SharedMemory *SHM;
```

```
int sid = shmget ( 23412, sizeof( SharedMemory), 0 ); // Assume already created.
SHM = shmat( sid, NULL, 0 );
```



```
// Or  
SharedMemory &shm = *SHM;
```

Then can reference duck by "SHM->duck" or by "shm.duck".

NOTE: It is very difficult to put STL objects in shared memory. Why??

How do we protect against multiple versions of the shared memory structure?

We will discuss in class the problem of needing an agreement between programs that use shared memory on the structure of shared memory.

Always remember, for this course, to clean up any shared memory that you have allocated.

Lab 10

Write a program to demonstrate how race conditions can occur with shared memory. Your program will do the following:

1. It will allocate a shared memory segment that contains an integer variable and set it to zero.
2. It will create a clone that will start adding 1 to the shared memory segment's integer 100,000,000 times.
3. The parent process will also add 1 to the shared memory segment's integer 100,000,000 times.
4. The parent will make sure that the child process has completed and then it will display the value of integer in shared memory.
5. The parent will then remove the shared memory segment.

Lab 11 Skip F2025

Write a program to implement a stream oriented queue using shared memory. Make the queue 100K. You should have two functions:

```
int sendQueue( char *buff, int buffsz ); Returns -1 for error and 0 if successful.  
int readQueue(char *buff, int buffsz); Returns the number of bytes read and -1  
if error.
```

Why not a bool return value for the first function? What kind of errors must we handle? How do we set up the queue structure? What are the risks?

Let's show a risk in using shared memory by creating a race condition. How would we do this? We will do it in class.

Final Thought 1

Vectors, maps, string, etc do not work with shared memory. Why? They can actually work, with some pain. So, it is time to go back to data structures. (Yeah!!)

Final Thought 2

There are alternative ways of having shared memory. One of these is memory maps. I will say a little about these in class. We talk about these later if there is time. They are considered more modern.

Final Thought 3

Play with the example, you will definitely see this again.

Message Queues

Message queues provide another means of inter-process communications that is independent of the file system. Our textbook (Stevens) is not a fan of message queues. It claims that they are too complicated. They are a little tricky, but I like them sometimes for the following reasons:

1. They are record oriented.
2. They do not require family relationship among communicating processes. It uses keys to establish relationships.
3. Allow multiple processes to write to and read from the queues in a manner that is thread safe.
4. Allows prioritizing of messages.
5. Allows directed messages.

As you will see later, you can also get into trouble using these.

System calls to support messages queues: `msgget`, `msgsend`, `msgrcv`, and `msgctl`. Some of the names should sound familiar from the discussion we had of shared memory.

```
#include <sys/types.h>
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
int msgget ( key_t key, int msgflg )
```

Returns the message queue ID associated with the key.
key has been discussed before. It identifies this IPC tool.

msgflg may be IPC_CREAT, IPC_PRIVATE, IPC_EXCL, MSG_R, MSG_W. The permission flags work in a manner similar to those used for shmget. For Linux use SHM_R and SHM_W or numeric values for permissions. I prefer using an octal number.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgsnd ( int msgqid, const void *msgp, size_t msgsz, int msgflg)
```

Returns: 0 if successful and -1 if fails.

msgqid - ID for the message queue.
msgp - pointer to the message. The message is in the format:

```
struct msgbuf{
    long mtype; /* Positive int describing message. */
    char mtext[1]; /* The message data. */
};
```

msgsz - the size of the data area to be sent. This value does not include the message type.
Msgflg - flags. IPC_NOWAIT means nonblocking. (errno = EAGAIN if would block.)

```
int msgrcv ( int msqid, void *msgp, size_t msgsz, long msgtype, int msgflg)
```

Returns: size of data portion of message if successful and -1 if fails.

msgqid - ID for the message queue.

msgp - pointer to the message. The message is in the format:

```
struct {
    long mtype; /* Positive int describing message. */
    char mtext[1]; /* The message data. */
}
```

msgsz - maximum size data area that we are ready to receive.

msgtype - specifies which message we want.

msgtype == 0 Want any message.

msgtype > 0 Want the first message with mtype == msgtype.

Msgtype < 0 Want the first message with mtype <= |msgtype|

msgflg - flags. IPC_NOWAIT means nonblocking. errno == ENOMSG if queue empty.

Talk of how we can use msgtype for client server applications. Talk of priority messages. msgctl system call. Used to perform various operations on a message queue.

```
#include <sys/types.h >
#include <sys/ipc.h>
#include <sys/msg.h>
int msgctl( int msgqid, int cmd, struct msqid_ds *buf );
```

Returns 0 if successful and -1 if fails.

cmd - IPC_STAT, IPC_SET (can set various elements associated with the message queue. E.g. the permissions), and IPC_RMID

Notes on Message Queues:

1. [They continue to live until they are explicitly deleted](#). This includes the messages that are in them. This can be a problem. Why?
2. Message queues have a size limit and a surprisingly small one. Typically, 16K for Linux and 4k for Solaris. (SUN's version of UNIX) I guess the pathetic size shows that SUN does not consider these important. [We will test the size in class](#).
3. Individual messages have size limits. 8K for Linux and 2K for Solaris.
4. Using positive type values in msgrcv for msgtype could dangerous. We will discuss why in class
5. Usually faster than pipes but not dramatically so. The book's statistics show this . They send 100,000 blocks of 2K and get 4.30 seconds for pipes and 4.16 seconds for message queues.

Example concept. We will discuss in class how two message queue can be used to support the interaction between a set of clients and a server. Note how we had better block all signals.

See examples. [send.cpp](#), [rec.cpp](#)

Lab 11

Write two programs. RCsend and RCreceive. Both will have a file name as a command-line argument. RCSend will use messages queues to send the contents of

its file to RCreceive. RCreceive will record in its file in its command line. Rcsend and RCreceive will terminate when they have completed their tasks. Think about how to do this.

Semaphores (Note: for simple tasks, there are better tools. We will lightly cover this.)

Discuss race conditions. Show how they can happen. We will write code to cause some race conditions in class. Please try and think of some way to cause this problem. (I put this comment here in case I forgot to discuss race conditions earlier.)

Semaphores are used to control access to shared resources. Idea for semaphores due to Dijkstra. He introduced the concept semaphores. These are global variables used for access control. He further introduced the P and V operators. The P and V operators are defined (not implemented) as follows:

```
void P(sem)
{
    while ( sem < 1 ) ; //Wait until sem >= 1
    sem--;
}
void V(sem)
{
    sem++;
}
```

Do example on how to protect a critical area of code.

Discuss why the above implementation will not work. Semaphores usually live in the operating system or have special machine language instructions to guarantee that they really work.

|

Linux semaphores are a more general and more confusing implementation of semaphores. Care has to be taken with semaphore in that there is some risk associated with them. There are alternatives to semaphores. These are referred to as mutexes. They are simpler and much more efficient when working with threads. They will be discussed later in this course.

The following are the system calls for semaphores.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
int semget( key_t key, int nsem, int semflg );
```

Returns: the semaphore ID for the set of semaphores. -1 if error.
key has been discussed before. It identifies this IPC tool. You can use IPC_PRIVATE to create private semaphores.
tool.

nsem - the number of semaphores requested. Most other operating systems only allow you to request one semaphore at a time.

Semflg - specifies options: IPC_CREAT, IPC_EXCL, and permissions. Same as other calls.

Note: semaphores in a set are referenced by the numbers 0 to nsems - 1.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
int semop( int semid, struct sembuf sops[], int nsops )
```

returns 0 if successful and -1 if fails.

semid - the ID of the semaphore set.

nsops - the number of semaphore operations (remember we can have multiple semaphores.)

sops - the operations to be performed on each of the semaphores. These are described by the structure:

```
struct sembuf {
    short sem_num; /* semaphore number */
    short sem_op; /* semaphore operation */
    short sem_flg; /* operation flags, IPC_NOWAIT, SEM_UNDO. */
};
```

sem_op - an integer that describes the semaphore operation:

- > 0 means increment semaphore by sem_op.
- == 0 wait until the semaphores value becomes zero.
- < 0 if |sem_op| <= semaphore value, add to semaphore. Otherwise, block until it is..

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
int semctl( int semid, int semnum, int cmd, union semun arg)
```

Returns: -1 if fails, 0 or command specific value.

```
union semun {
    int val;
    struct semid_ds *buf;
    ushort_t *array;
} arg;
```

cmd - command: IPC_STAT, IPC_SET, IPC_RMID, GETVAL (returns value of semaphore), SETVAL (sets value of semaphore to arg.val), SETALL set all semaphores to arg.array, etc.

Examples [semtest.c](#) [semtest2.c](#) These examples will be done in class.

Notes:

1. There is a limit on how many semaphore you can have on the system. It is relatively small. In Solaris you can only have 10 sets. For Linux this is not an issue. You can have up to 32,000 semaphores arrays and 32,000 semaphores per array. Even with much smaller limits, this is not a problem. Our system does have big limits.
2. The SEM_UNDO is pretty cool. It allow a recovery from a program that terminates. Otherwise an aborted program can leave all others hanging on a semaphore.

Lab 11 Not now

Create a shared memory segment for a long int variable initially 0. Then us fork to schedule ten processes. Each of the process will add 10 onto the variable, a sufficient number of times to see race conditions. Then display the final result. Rewrite a second version of the program using semaphores to protect the variable. Be care not to make the summation too big.

